PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

CONSUMO Y DOCUMENTACIÓN DE UNA API CON REACT

Presentado a: Alexandra guevara

Por Aprendiz: **Luis fernando conejo**

Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **XX** de **XXXX** del año **XXXX**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

1. Introducción y Objetivo

¿Qué se desarrolló?

Se desarrolló una aplicación web interactiva del lado del cliente (Frontend) utilizando React y Vite, diseñada para comunicarse con una API RESTful construida previamente en Node.js y Express (Backend). La interfaz permite consultar diferentes endpoints informativos mediante botones de interacción y enviar datos dinámicos a través de un formulario con validaciones.

¿Qué necesidad resuelve?

Resuelve la necesidad de integrar una interfaz visual de usuario con un servidor backend, permitiendo desacoplar la lógica del servidor de la experiencia visual del usuario. Facilita la consulta de información en tiempo real y el envío estructurado de solicitudes (como registros o formularios) de manera asíncrona, sin necesidad de recargar la página web.

Relación entre React y Express

La relación se basa en el modelo cliente-servidor mediante el protocolo HTTP.

Express (Backend): Actúa como el servidor central. Se encarga de procesar las peticiones HTTP (GET, POST), aplicar validaciones sobre los datos recibidos, interactuar con la lógica de negocio y responder estructuradamente en formato JSON.

React (Frontend): Actúa como el cliente. Es la interfaz gráfica que visualiza el usuario en su navegador. Utiliza funciones asíncronas (fetch y async/await) para realizar peticiones hacia las rutas expuestas por Express, recibe las respuestas JSON y actualiza dinámicamente los componentes visuales en pantalla mediante el uso de estados (useState).

2. Instalación y Configuración

¿Para qué sirven Node.js y npm?

Node.js: Es un entorno de ejecución de JavaScript basado en el motor V8 de Google Chrome. Permite ejecutar código JavaScript fuera del navegador, siendo la base fundamental para levantar servidores web (como el backend en Express) y herramientas de desarrollo.

npm (Node Package Manager): Es el gestor de paquetes oficial de Node.js. Permite instalar, actualizar y administrar bibliotecas, dependencias y herramientas de terceros necesarias para el desarrollo del proyecto (como Vite, librerías de CORS, etc.).

Cómo se creó el proyecto con Vite

El proyecto de Frontend se inicializó ejecutando los siguientes comandos en la terminal Git Bash:

Se ejecutó el comando de creación de Vite:

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Bash

*npm create vite@latest tallerr

Durante el asistente interactivo de configuración, se seleccionaron las siguientes tecnologías:

*Framework: React.

*Variante / Lenguaje: JavaScript.

Herramientas adicionales: ESLint (para el análisis estático del código).

Una vez creado el directorio, se ingresó a la carpeta del proyecto y se instalaron las dependencias necesarias mediante la consola integrada de Visual Studio Code:

*Bash

*npm install

Finalmente, el entorno de desarrollo local del frontend se puso en marcha ejecutando:

*Bash

*npm run dev

Esto habilitó un servidor local de desarrollo (por defecto en http://localhost:5173).

¿Qué función cumple cada archivo importante?

index.html: Es la plantilla HTML principal de la aplicación donde se inyecta la aplicación de React.

src/main.js (o main.jsx): Es el punto de entrada de JavaScript que conecta el componente raíz <App/> con el DOM del navegador.

src/App.jsx: Es el componente principal o contenedor que organiza y renderiza los subcomponentes de la aplicación (Inicio, Informacion, Solicitudes).

package.json: Archivo de configuración que almacena los metadatos del proyecto, las dependencias instaladas y los scripts de ejecución (npm run dev).

¿Cómo se verificó que el backend estuviera funcionando?

Antes de poner a correr el frontend en React, nos aseguramos de que el servidor de Express estuviera activo y rodando sin problemas en el puerto 3000. Para comprobar que respondía bien, abrimos las rutas directamente en el navegador o hicimos pruebas rápidas con **Insomnia**, validando que el servidor devolviera los datos en formato JSON y el código de estado HTTP 200 (todo OK).

3. Desarrollo de los Componentes

3.1. Componente Inicio.jsx

```
import { useState } from "react"



export default function Inicio(){
   const[inicio,setInicio]=useState("")
```

```
    async function Api(){
       let respuesta =await fetch("http://localhost:3000/inicio")
       let datos = await respuesta.json()
       console.log(datos)
       setInicio(datos.mensaje)
```

```
   }
   return(
       <div>
           <p>{inicio}</p>
           <button onClick={Api}>llamar Api</button>
```

```
       </div>
   )
}
```

Qué responsabilidad cumple: Consultar de forma rápida el mensaje de bienvenida expuesto por el servidor backend.

Qué endpoint consume: /inicio

Qué método HTTP utiliza: Método GET.

Cómo se ejecuta la petición: Se activa al hacer clic en el botón mediante el evento onClick, el cual ejecuta la función asíncrona Api().

Cómo recibe y procesa la respuesta: La función espera la respuesta del servidor con await fetch() y la transforma a formato objeto usando await respuesta.json().

Cómo muestra la información: Extrae la propiedad datos.mensaje y la guarda en el estado inicio mediante setInicio(), pintándola en el párrafo <p>.

Qué errores pueden presentarse y cómo los manejaste: Si el servidor está apagado o no hay conexión, la petición genera un fallo de red a nivel de navegador.

3.2. Componente Informacion.jsx

```
import { useState } from "react";


export default function Informacion() {
  const [servicio, setServicio] = useState("");
  const [descripcion, setDescripcion] = useState("");
  const [version, setVersion] = useState("");
```

```
  async function Api() {
    let respuesta = await fetch("http://localhost:3000/informacion");
    let datos = await respuesta.json();
    console.log(datos);
```

```


    setServicio(datos.servicio);
    setDescripcion(datos.descripcion);
    setVersion(datos.version);
```

```
  }
```

```
  return (
    <div>
      <button onClick={Api}>Consultar Información</button>
      <p><strong>Servicio:</strong> {servicio}</p>
      <p><strong>Descripción:</strong> {descripcion}</p>
      <p><strong>Versión:</strong> {version}</p>
    </div>
  );
}
```

Qué responsabilidad cumple: Obtener los datos descriptivos de la API para mostrarlos organizados por propiedades en la interfaz.

Qué endpoint consume: /informacion

Qué método HTTP utiliza: Método GET.

Cómo se ejecuta la petición: Se ejecuta al presionar el botón "Consultar Información" mediante un evento onClick.

Cómo recibe y procesa la respuesta: Procesa la respuesta con await respuesta.json() y asigna cada campo (datos.servicio, datos.descripcion, datos.version) a sus estados correspondientes.

Cómo muestra la información: Usa tres estados independientes (servicio, descripcion, version) que se actualizan al tiempo y renderizan su contenido en sus respectivos párrafos <p>.

Qué errores pueden presentarse y cómo los manejaste: Si hay problemas de comunicación con la API, el navegador notifica el error en consola sin bloquear la ejecución del resto de la interfaz.

3.3. Componente Solicitudes.jsx

```
import { useState } from "react"


export default function Solicitudes() {

  // 1. Estados para capturar lo que el usuario escribe en los inputs
```

```
const [nombre, setNombre] = useState("")
const [apellido, setApellido] = useState("")
const [correo, setCorreo] = useState("")
```

```
// 2. Estados para guardar y mostrar la respuesta que devuelve el backend
const [dato_enviado, setDatoEnviado] = useState("")
const [dato_apellido, setDatoApellido] = useState("")
const [dato_correo, setDatoCorreo] = useState("")
const [mensaje, setMensaje] = useState("")
```

```
async function guardarApi() {
    // Hacemos la petición POST al backend
    let resultado = await fetch("http://localhost:3000/solicitudes", {
        method: 'POST',
        body: JSON.stringify({
            nombre: nombre,
            apellido: apellido,
            correo: correo
        }),
        headers: {
            'content-type': 'application/json'
        }
    })

    // Convertimos la respuesta a objeto JSON
    let datos = await resultado.json()
    console.log(datos)
```

```
    // Guardamos las respuestas del backend en el segundo grupo de estados
    setDatoEnviado(datos.dato_enviado)
    setDatoApellido(datos.dato_apellido)
    setDatoCorreo(datos.dato_correo)
    setMensaje(datos.mensaje)
  }
```

```
  return (
    <div>
      <h2>Formulario de Solicitudes</h2>

      // campos para que el usuario escriba
      <div>
        <input
          placeholder="nombre"
          value={nombre}
          onChange={(e) => setNombre(e.target.value)}
        />
      </div>
      <div>
        <input
          placeholder="apellido"
          value={apellido}
          onChange={(e) => setApellido(e.target.value)}
        />
      </div>
```

```
        <div>
          <input
            placeholder="correo"
            value={correo}
            onChange={(e) => setCorreo(e.target.value)}
          />
        </div>

        <button onClick={guardarApi}>aceptar</button>

        //respuestas del servidor
        <p><strong>Nombre guardado:</strong> {dato_enviado}</p>
        <p><strong>Apellido guardado:</strong> {dato_apellido}</p>
        <p><strong>Correo guardado:</strong> {dato_correo}</p>
        <p><strong>Respuesta servidor:</strong> {mensaje}</p>
      </div>
  )
}
```

Qué responsabilidad cumple: Capturar los datos ingresados en tres campos de texto (nombre, apellido y correo) y enviarlos mediante una petición POST al servidor Express.

Qué endpoint consume: /solicitudes

Qué método HTTP utiliza: Método POST.

Cómo se ejecuta la petición: Se activa al hacer clic en el botón "aceptar" mediante el evento onClick, ejecutando la función asíncrona guardarApi().

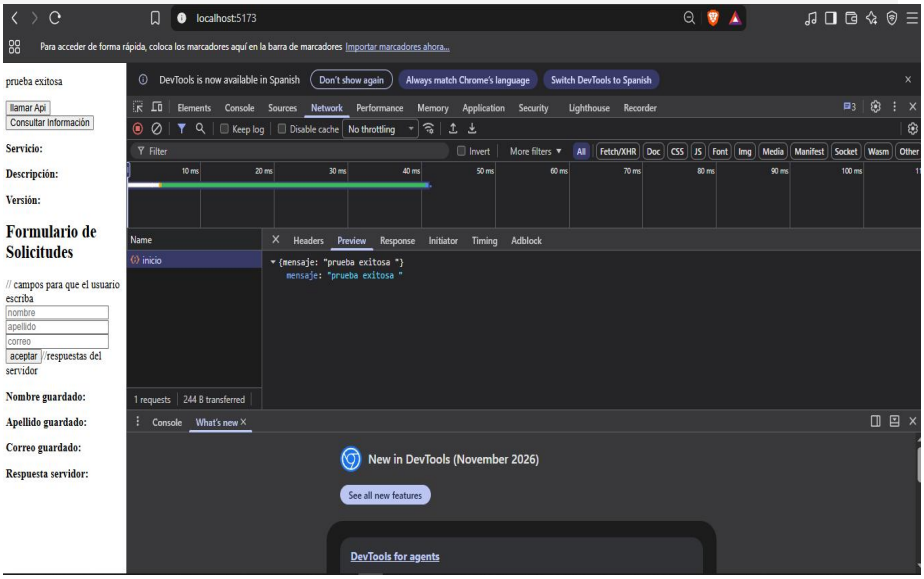
Cómo recibe y procesa la respuesta: Envía los datos en el cuerpo de la petición con JSON.stringify() y configurando el encabezado 'content-type': 'application/json'. Al recibir la respuesta de Express, la convierte con .json() y actualiza los cuatro estados de respuesta (setDatoEnviado, setDatoApellido, setDatoCorreo, setMensaje).

Cómo muestra la información: Renderiza directamente en pantalla los valores que devuelve el backend, permitiendo confirmar en tiempo real lo que Express procesó y respondió.

Qué errores pueden presentarse y cómo los manejaste: Si la API rechaza la información por alguna validación del backend, el mensaje de error o rechazo devuelto por Express se guarda en el estado mensaje y se visualiza directamente en la pantalla.

3.4 Pruebas obligatorias

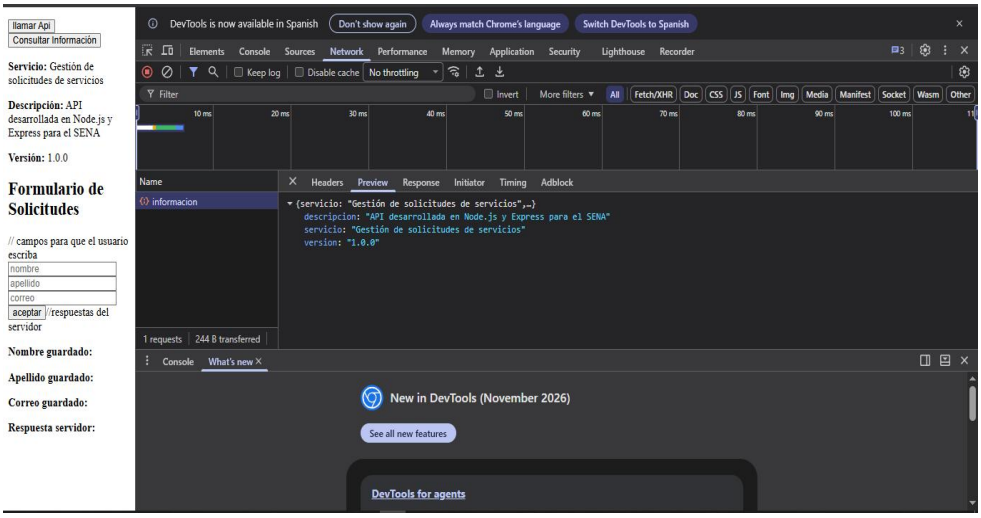
1. Consultar /inicio Mostrar el mensaje recibido de Express.



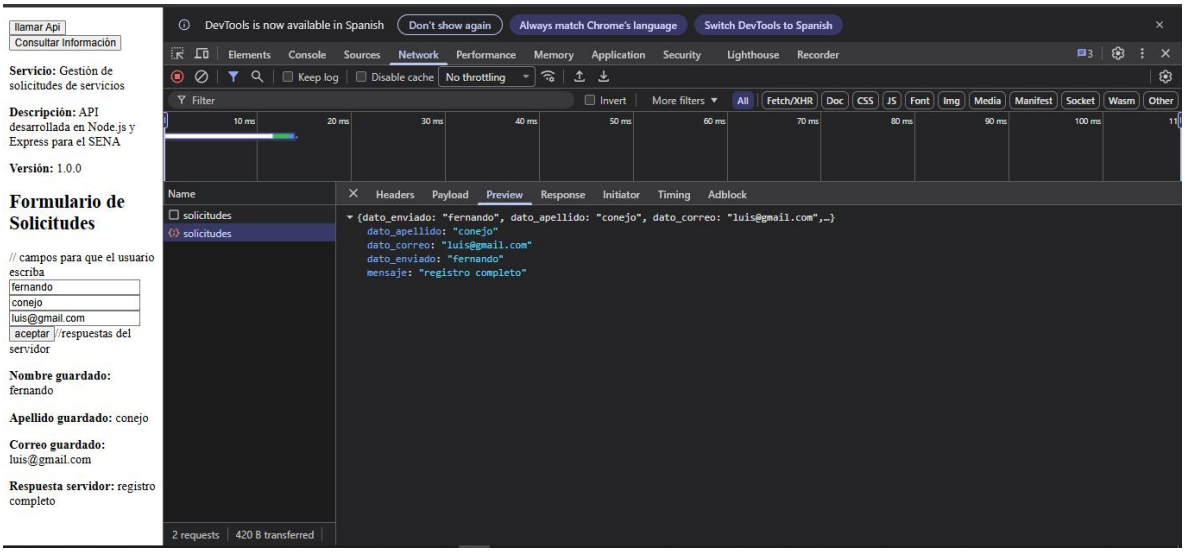
2

2.Consultar /informacion Mostrar la información devuelta por la API.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

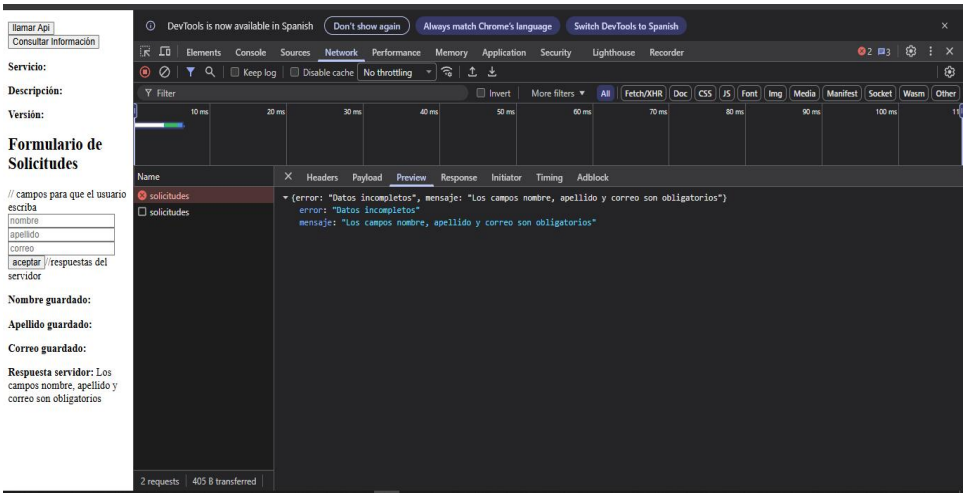


3.Enviar un formulario completo Mostrar la respuesta de la API.

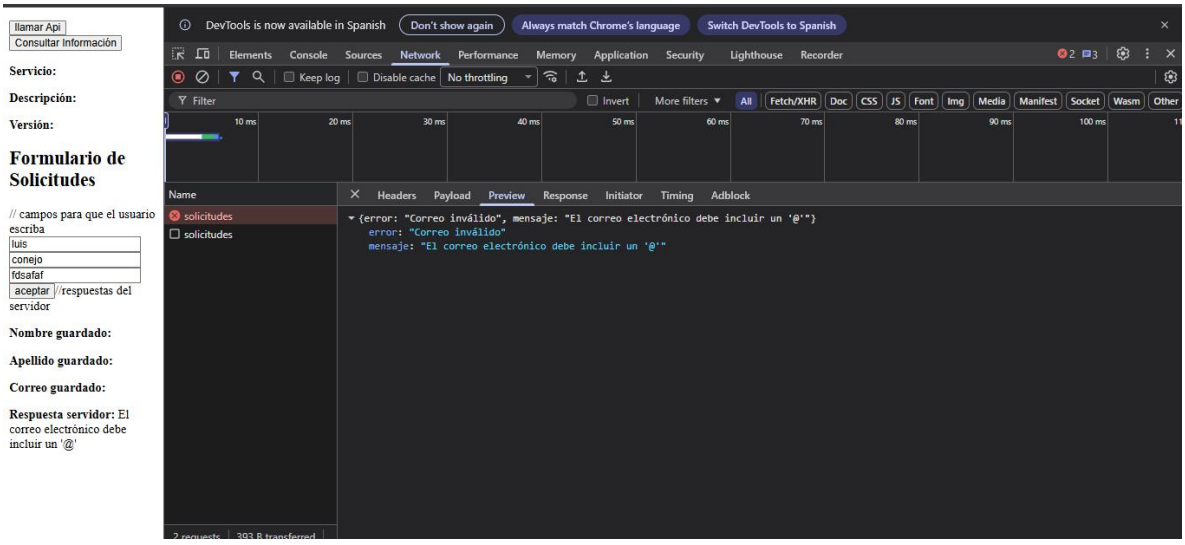


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

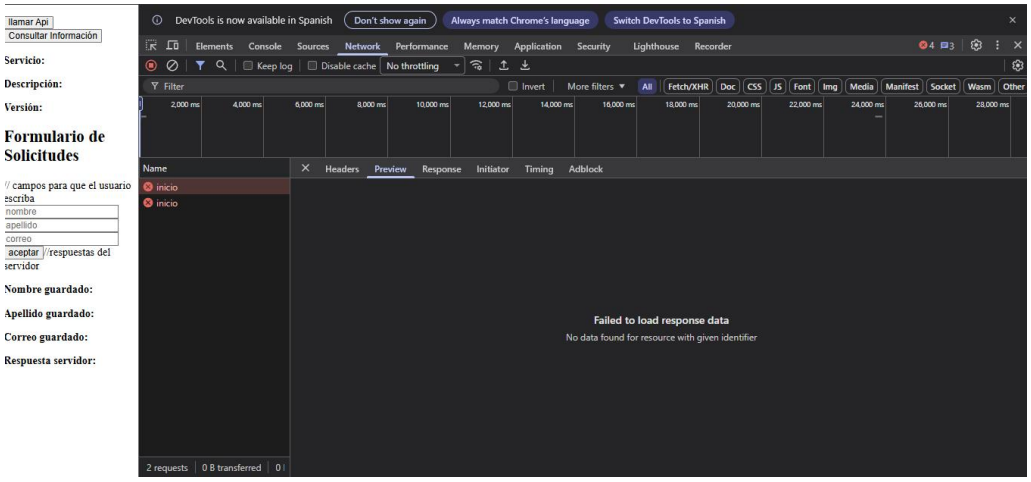
4.Enviar campos vacíos	Mostrar las validaciones que correspondan.



5.Enviar un correo inválido	Mostrar el rechazo de la API o la validación implementada en el frontend.



6.Apagar temporalmente Express y realizar una consulta	Mostrar un mensaje de error de conexión.



7.Enviar una solicitud con datos inválidos	Mostrar el mensaje de error devuelto por el backend.

3.5. Investigación obligatoria sobre consumo de API

1.¿Qué es una API y para qué sirve en una aplicación web?

¿Qué es?: Es como un puente o un cartero que permite que dos sistemas hablen entre sí. En una app web, conecta la parte visual (lo que ve el usuario) con el servidor y la base de datos.

¿Para qué sirve?: Para que tu página web pueda pedir datos (como consultar productos o usuarios) o guardarlos (como registrar un formulario) sin necesidad de recargar toda la página.

2.¿Qué diferencia existe entre frontend y backend?

Frontend: Es la cara visible de la aplicación, lo que el usuario ve y toca en el navegador (botones, formularios, colores, diseñados con React, HTML, CSS).

Backend: Es la parte de atrás, el cerebro que corre en el servidor (con Node.js y Express). Maneja la lógica, procesa los datos y se conecta con la base de datos.

3. ¿Qué significa consumir una API?

Significa que el frontend hace una petición (un llamado) hacia el servidor o una API externa para pedirle información o enviarle datos, y luego recibe una respuesta para mostrarla en la pantalla.

4. ¿Qué función cumple fetch()?

Es una función nativa de JavaScript que sirve precisamente para hacer peticiones HTTP (como GET o POST) hacia un servidor y traer o enviar datos de forma sencilla sin instalar librerías externas.

5. ¿Por qué se utilizan async y await en una petición HTTP?

Porque las peticiones a un servidor toman un tiempo (milisegundos o segundos) en ir y volver. async y await sirven para decirle a JavaScript que espere a que el servidor responda antes de seguir ejecutando el código, evitando que la app falle o se quede congelada.

6. ¿Qué es una promesa en JavaScript?

7. ¿Para qué sirve response.json()?

Cuando el servidor responde a un fetch, los datos crudos vienen en formato de texto plano o stream. Usar .json() convierte esa respuesta cruda en un objeto de JavaScript fácil de leer y manipular en tu código.

8. ¿Qué diferencia existe entre una petición GET y una petición POST?

GET: Se usa solo para **consultar o leer** datos del servidor (por ejemplo, traer la lista de usuarios). No envía información oculta.

POST: Se usa para **enviar datos nuevos** al servidor (por ejemplo, registrar un formulario o crear un usuario nuevo).

9. ¿Qué función cumplen las cabeceras HTTP y por qué se utiliza Content-Type: application/json?

10. ¿Por qué se utiliza JSON.stringify() al enviar datos en una petición POST?

Porque los servidores web solo entienden texto JSON.stringify() agarra un objeto de JavaScript (las variables o datos del formulario) y lo convierte en un texto con formato JSON para que pueda viajar correctamente en el cuerpo de la petición.

11. ¿Qué diferencia existe entre un error de conexión y una respuesta HTTP con código 400?

Error de conexión: Ocurre cuando el servidor está totalmente apagado o no hay internet, por lo que la petición ni siquiera llega a ejecutarse.

Código 400 (Bad Request): Significa que el servidor sí estaba prendido y recibió la petición, pero te rechaza porque los datos que mandaste están incompletos o mal estructurados.

12. ¿Qué función cumplen useState y los eventos onClick y onSubmit?

useState: Es un gancho de React para guardar y actualizar datos o estados dentro del componente (como lo que escribe el usuario o las respuestas del servidor).

onClick: Es un evento que detecta cuando el usuario hace clic en un botón.

onSubmit: Es un evento que se activa cuando se envía un formulario.

13. ¿Qué es CORS y por qué puede afectar la comunicación entre React y Express?

14. ¿Qué ocurre si el frontend envía propiedades diferentes a las que espera el backend?

El backend no sabrá qué hacer con esos datos extra o faltantes. Por lo general, los ignorará, arrojará un error de validación o fallará al intentar leer campos que llegan como undefined.

15. ¿Por qué no se debe considerar exitosa toda petición solamente porque fetch() devolvió una respuesta?

Porque fetch() solo se considera "exitoso" si logró comunicarse con el servidor. Si el servidor te responde con un error controlado (como un código 400 o 500), el fetch igual se completa, pero los datos que trae adentro son un mensaje de fallo. Por eso siempre toca revisar manualmente el estado o si la respuesta trae algún indicador de error.

3.6 Conclusiones y referencias

Primero entendí cómo conectar un React con un servidor backend. Lo del manejo del fetch y async await me confundo un poco aún, pero entendí mejor. Durante el desarrollo no se me presentaron errores graves porque entendí la estructura inicial del proyecto, lo que me facilitó un poco la creación de los componentes. En general, entendí casi todo.

Referencias

MDN Web Docs: *Usando Fetch*. Disponible en: https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

React Documentation: *State: A Component's Memory*. Disponible en: https://react.dev/learn/state-a-components-memory

Express Documentation: *Routing in Express*. Disponible en: https://expressjs.com/es/guide/routing.html